

System Architecture Specification: Full-Stack Deep Dive

เอกสารสถาปัตยกรรมระบบอย่างละเอียด: Frontend (Next.js 15), Backend (FastAPI), GraphRAG (LangGraph + Neo4j + Qdrant), Data Flow และ State Machine

Version: 1.0.0 (Production Architecture)	Generated Date: August 2026	Classification: System Architecture
Core Stack: Next.js 15, FastAPI, Neo4j, Qdrant	AI Engine: LangGraph Agentic RAG + OpenRouter	Intelligence: MITRE ATT&CK; STIX 2.1

1. ภาพรวมระบบและหลักการออกแบบ (Executive Summary & Architecture Vision)

CyberCase Intelligence Framework เป็นระบบ Full-Stack Agentic RAG ที่ออกแบบมาเพื่อการสืบสวนและวิเคราะห์เหตุการณ์ภัยคุกคามทางไซเบอร์ (Cybersecurity Incident Analysis) โดยทำการจับคู่พฤติกรรมการโจมตีเข้ากับฐานความรู้ MITRE ATT&CK; (STIX 2.1) แบบ 2-Hop Graph Traversal ร่วมกับการสืบค้นเชิงความหมาย (Dense Vector Search) และมีระบบจัดเก็บประวัติการวิเคราะห์แบบ Immutable Case State เพื่อให้ทุกการวิเคราะห์สามารถตรวจสอบย้อนกลับ (Auditability) ได้อย่างสมบูรณ์

หลักการสถาปัตยกรรม	รายละเอียดการนำไปใช้งาน (Implementation Details)
Single Source of Truth	PostgreSQL เป็นแหล่งข้อมูลเดียวที่เป็น Authoritative State สำหรับแชท (Threads), ข้อความ (Messages), การประมวลผล (Runs), และประวัติสถานะเคส (CaseStateVersions) โดย Frontend จะไม่มี local state ที่ขัดแย้งกับฐานข้อมูล
Decoupled GraphRAG	`rag_service` ทำงานเป็น Microservice อิสระ ทำหน้าที่ประมวลผลการค้นหา กราฟ และ AI เท่านั้น โดย Backend เป็นผู้ดูแล Clarification State, Follow-up Gating และ Snapshot Management
Immutable Case State DAG	ทุกการสกัดข้อมูลและอัปเดตเคสจะสร้างเวอร์ชันใหม่ (`CaseStateVersion`) เสมอ มี parent lineage ชัดเจน ป้องกันการเขียนทับ (In-place Mutation) และรองรับการทำ Audit Trail
Asynchronous Run Lease	ระบบ Chat Ingestion ทำงานแบบ Asynchronous Job (HTTP 202 Accepted) ควบคุมผ่าน Worker Lease (`RUN_LEASE_DURATION` 60s) ป้องกัน Deadlock และ Concurrency Collision

2. ผังโครงสร้างระดับสูง (High-Level System Topology & Services)

โครงสร้างระบบแบ่งออกเป็น 3 เลเยอร์หลัก พร้อมฐานข้อมูลเฉพาะทาง 4 ชนิด เชื่อมโยงผ่าน Docker Compose:



3. สถาปัตยกรรมฝั่ง Frontend (Next.js 15 + React 19)

Frontend ถูกสร้างบนสถาปัตยกรรม Next.js 15 App Router ร่วมกับ React 19 และ Tailwind CSS 4 โดยเน้นความเป็น Cognitive Forensic Workspace ที่สามารถประมวลผลและแสดงผลข้อมูลการสืบสวนที่มีความซับซ้อนสูงได้อย่างราบรื่น

3.1 โครงสร้างโฟลเดอร์และส่วนประกอบหลัก (Component Architecture)

ไดเรกทอรี / โมดูล	หน้าที่และความรับผิดชอบ (Role & Responsibilities)
<code>`src/app/chat/page.tsx`</code>	Main Entrypoint ของหน้าแชทหลัก จัดการ Life cycle, การเลือก Thread, และทำ Data Fetching เริ่มต้น
<code>`src/components/ChatWorkspace.tsx`</code>	ศูนย์กลางควบคุม Layout แบ่งเป็น 3 ส่วน: Sidebar, Conversation Feed, และ Intelligence Inspector Tabs พร้อมคุม Master State
<code>`src/components/conversation/`</code>	โมดูลข้อความ: <code>`MessageBubble`</code> , <code>`MessageList`</code> , <code>`Composer`</code> (กล่องพิมพ์ข้อความที่ปรับตามสถานะ), <code>`ActionSelector`</code> (เลือกเจตนา: New Incident, Add Info, Ask), และ <code>`FollowupBanner`</code>
<code>`src/components/analysis/`</code>	โมดูลวิเคราะห์เคส: <code>`CaseOverview`</code> (สรุปผลการวิเคราะห์), <code>`MitreTechniqueList`</code> (ตารางเทคนิค MITRE พร้อม Confidence Score), <code>`ExtractedEntities`</code> (Entity Badges), และ <code>`RawEvidenceViewer`</code>
<code>`src/components/timeline/` & <code>`relationships/`</code></code>	แสดงลำดับเหตุการณ์การโจมตี (Chronological Timeline Cards) และแผนผังกราฟความสัมพันธ์ระหว่าง Actor, Tool, และ Victim
<code>`src/components/report/`</code>	โมดูลรายงาน: แสดงตัวอย่าง Markdown Report, ตัวเลือกดาวน์โหลด PDF (<code>`pdf`</code>), และประวัติเวอร์ชันรายงาน
<code>`src/lib/api.ts`</code>	Type-safe API Client เชื่อมต่อ Backend Endpoint พร้อมระบบจัดการ Error, AbortSignal และ Retry Mechanism

3.2 วงรอบการทำงานและการจัดการความสอดคล้องของ State (Polling & Concurrency Guard)

เนื่องจาก Backend ประมวลผลแบบ Asynchronous Background Task ฝั่ง Frontend จึงมีกลไก Polling และ Concurrency Guard ที่รัดกุม:

- Lease-Aware Polling (1s Interval): หลังส่งข้อความและได้ `202 Accepted` Frontend จะเริ่ม Poll `GET /api/v1/chats/{thread\_id}` ทุก 1 วินาที โดยตรวจสอบสถานะ `status` (`processing` \$ 0\$ Poll ต่อ, `awaiting\_followup` \$ 0\$ เปิดกล่องถามคำถามชี้แจง, `idle`/`answered` \$ 0\$ หยุด Poll และเรนเดอร์คำตอบ)
- Dual Concurrency Guard: ป้องกันปัญหา Race Condition เมื่อผู้ใช้สลับ Thread อย่างรวดเร็ว โดยใช้:
 - AbortController: ยกเลิก HTTP Request/Polling ของ Thread เดิมทันทีที่มีการสลับ Thread
 - selectionGenerationRef: ตัวนับลำดับการเลือก (Generation Monotonic Counter) เพื่อปฏิเสธการตอบกลับที่มาช้ากว่า (Late Response) ไม่ให้เขียนทับ State ของ Thread ปัจจุบัน
- Idempotency Key Injection: ทุกข้อความที่ส่งจะสร้าง UUID เฉพาะเป็น `idempotency\_key` เพื่อป้องกันการส่งซ้ำกรณี Network Retry

4. สถาปัตยกรรมฝั่ง Backend (FastAPI + SQLAlchemy Async)

Backend ทำหน้าที่เป็น Orchestrator กลางของทั้งระบบ ควบคุมตั้งแต่ Data Ingestion, Background Worker Lease, การสกัดข้อมูล Structured Case State, การเรียก GraphRAG, ตรวจสอบ Clarification Gating, จนถึงการบันทึกผลลัพธ์ลง PostgreSQL

4.1 โครงสร้างเลเยอร์ของ Services (Backend Modular Services)

โมดูล Service	คำอธิบายการทำงานเชิงลึก (In-Depth Technical Role)
`services/workflow/`	Worker & Pipeline Core: จัดการ Run Lease Lock (`worker.py`), รัน Execution Pipeline 5 ขั้นตอน (`pipeline.py`), และ Mapping ผลลัพธ์ (`outcome.py`)
`services/case_state/`	Case State & Mutation: จัดการ Immutable DAG Versions, คำนวณ State Delta จากข้อความใหม่ (`mutator.py`), และแปลง Case State เป็นข้อความค้นหาที่กระชับสำหรับ RAG (`projector.py`)
`services/extraction/`	Structured Extraction: ทำหน้าที่สกัด Entities, Evidence, Timeline, และ Relationships จากข้อความดิบของผู้ใช้ ด้วย JSON Schema Validation และ Entity Normalization
`services/case_analysis/`	Grounded Case Overview: ทำงานร่วมกับ LLM เพื่อสร้างบทวิเคราะห์เชิงเทคนิคที่ Grounded บนข้อมูลจาก GraphRAG และ Case State พร้อมสร้าง Analysis Trace และ Claims
`services/followup/`	Gap Analysis & Clarification Gate: วิเคราะห์ช่องว่างของข้อมูล (IOCs, Timestamps, Vectors) จัดลำดับความสำคัญ (Priority Queue) และตัดสินใจว่าจะถามคำถามเพิ่มเติมหรือจบเคส
`services/reports/`	Deterministic Report & PDF: สร้างรายงาน Markdown และเรนเดอร์เอกสาร PDF ผ่าน ReportLab จาก Snapshot ที่ถูก Freeze ไว้ พร้อมตรวจสอบความถูกต้องของ SHA-256 Hash
`services/clients/` & `llm/`	HTTP Client & Model Registry: จัดการการเชื่อมต่อ HTTP ไปยัง `rag_service` (`rag_client.py`) และระบบ Routing/Fallback ของ OpenRouter โมเดล LLM

4.2 ขั้นตอนการประมวลผล Background Pipeline (Execution Pipeline Lifecycle)

เมื่อฟังก์ชัน `process\_chat\_run` ใน `services/workflow/pipeline.py` ทำงาน จะผ่านขั้นตอนลำดับดังนี้:

```
[Step 1: Claim Lease] ChatRunWorker ทำการ Claim Lease บน ChatRun ID พร้อมใส่ Worker ID และตั้ง TTL (60s)
|
▼
[Step 2: Action Branch] แยก Flow ตาม Post-Answer Action:
├─▶ "initial_query": รัน Baseline Extraction -> สร้าง CaseStateVersion v1 -> ส่ง RAG Query
├─▶ "add_case_info": รัน Delta Extraction -> Apply Delta -> สร้าง CaseStateVersion vN -> ส่ง RAG Query
├─▶ "ask": ข้าม Extraction และ RAG -> นำ Case State ปัจจุบัน + Analysis Context มาตอบคำถาม Q&A; ทันที
|
▼
[Step 3: GraphRAG Query] แปลง Case State เป็น Focused Query -> เรียก POST http://rag_service:8001/query
|
└─▶ ได้ MITRE Techniques + Graph Context + Graph Reasoning
|
▼
[Step 4: Case Overview] เรียก Case Analysis LLM เพื่อสังเคราะห์ภาพรวมการโจมตี (Overview Narrative)
|
└─▶ พร้อมจับคู่ Evidence และสร้าง Atomic Claims
|
▼
[Step 5: Followup Gate] ประเมิน Gap Analysis: มีข้อมูลสำคัญขาดหายหรือไม่?
├─▶ ขาดข้อมูลสำคัญ (Gap Found) -> เปลี่ยนสถานะเป็น "awaiting_followup" + สร้าง Assistant Question
├─▶ ข้อมูลครบถ้วน (No Critical Gap) -> เปลี่ยนสถานะเป็น "idle" + คืนคำตอบสมบูรณ์ (Completed Answer)
|
▼
[Step 6: Persist Outcome] บันทึก Assistant Message, อัปเดต Thread Status, บันทึก CaseStateVersion, ปลด Run Lease
```

5. สถาปัตยกรรม GraphRAG และ Hybrid Retrieval (`rag\_service`)

`rag\_service` เป็นหัวใจสำคัญในการสืบค้นและจับคู่ข้อมูลจากห้องสมุดความรู้ โดยผสานพลังระหว่าง Dense Vector Search และ Graph Knowledge Expansion เข้าด้วยกันอย่างสมบูรณ์แบบ

องค์ประกอบ RAG	กลไกและเทคโนโลยีที่ใช้ (Technical Mechanism)
Dense Vector Search	ใช้โมเดล BAAI/bge-m3 (1024-dimensional embeddings, รองรับภาษาไทย-อังกฤษสมบูรณ์) จัดเก็บใน Qdrant Vector Database ทำการค้นหาเชิงความหมายด้วย Cosine Similarity
Graph Traversal	จัดเก็บโครงสร้าง MITRE ATT&CK; STIX 2.1 ใน Neo4j Graph Database ดึงความสัมพันธ์ระดับ 2-Hop Cypher Query เชื่อมโยง Tactics, Techniques, Sub-techniques, Mitigations, และ Threat Groups
Reciprocal Rank Fusion	ผสานผลลัพธ์จาก Vector Search และ Graph Centrality ด้วยอัลกอริทึม RRF (k=60) เพื่อคัดเลือกเทคนิคที่สอดคล้องกับพฤติกรรมของภัยคุกคามมากที่สุด
Cross-Encoder Reranker	ปรับปรุงลำดับความแม่นยำด้วยโมเดล Reranker `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1` ก่อนส่งเข้าสู่ Context Builder
LangGraph Agentic Loop	ควบคุม Flow ด้วย State Machine: แปลงภาษา (Cross-Lingual Thai $\rightarrow$ English), ตรวจสอบความเพียงพอของบริบท (Sufficiency Evaluator), และวนซ้ำขยายการค้นหา (Broaden Search Loop) สูงสุด 2 รอบ

6. โครงสร้างฐานข้อมูลและการจัดเก็บสถานะ (PostgreSQL Persistence Schema)

โครงสร้างตารางใน PostgreSQL ถูกออกแบบตามหลัก Relational Integrity พร้อม Foreign Key Constraints และ Cascading Deletes:

ชื่อตาราง (Table Name)	Primary Key / Constraints	หน้าที่และความหมายของข้อมูล (Description & Schema Role)
`chat_threads`	`id` (UUID) Check: status IN ('idle', 'processing', 'awaiting_followup', 'answered', 'failed')	เก็บบทสนทนาหลัก, ชื่อเคส (`title`), สถานะปัจจุบัน, `next_message_ordinal`, และ Foreign Key ชี้นำไปยัง `current_case_state_version_id`
`chat_messages`	`id` (UUID) FK: `thread_id` (CASCADE) Unique: (`thread_id`, `ordinal`)	เก็บข้อความทั้งหมดเรียงตาม `ordinal` พร้อม `role` (`user`, `assistant`, `system`), `content`, และ `metadata_json` (เก็บ Extraction, Gaps, Traces)
`chat_runs`	`id` (UUID) FK: `thread_id` (CASCADE) Check: status IN ('queued', 'running', 'completed', 'failed')	จัดการงานประมวลผล Background Worker เก็บ `worker_id`, `lease_expires_at`, `operation`, `post_answer_action`, และ Error Details
`case_state_versions`	`id` (UUID) FK: `thread_id` (CASCADE) Unique: (`thread_id`, `version`)	บันทึกประวัติสถานะเคสแบบ Immutable DAG เก็บ `state_json` (Entities, Evidence, Timeline, Relations), `delta_json`, และ `parent_version_id`
`chat_reports`	`id` (UUID) FK: `thread_id` (CASCADE) Unique: (`thread_id`, `version_number`)	เก็บรายงานฉบับสมบูรณ์ (Report Snapshot) พร้อม `source_snapshot_hash` (SHA-256), `report_json` (Structured Claims), และ Markdown Content
`rag_context_snapshots`	`id` (UUID) FK: `thread_id` (CASCADE)	จัดเก็บ Snapshot ของผลลัพธ์การสืบค้นจาก GraphRAG สำหรับการตรวจสอบความสอดคล้องและการทำ Audit

7. สรุป API Contracts และลำดับการส่งข้อมูล (End-to-End Workflow & REST Endpoints)

Backend ให้บริการ RESTful API ภายใต้ Prefix `/api/v1/chats` ตามสัญญาข้อตกลง (API Contracts) ดังนี้:

Method	Endpoint Route	Status Code	คำอธิบายการทำงาน (Function & Behavior)
--------	----------------	-------------	--

<code>`GET`</code>	<code>`/api/v1/chats`</code>	<code>`200 OK`</code>	เรียกดูรายการ Thread ทั้งหมด เรียงลำดับตาม <code>`updated_at`</code> ล่าสุด
<code>`POST`</code>	<code>`/api/v1/chats`</code>	<code>`201 Created`</code>	สร้าง Thread ใหม่ (กำหนดชื่อหรือใช้ค่าเริ่มต้น 'New chat')
<code>`GET`</code>	<code>`/api/v1/chats/{thread_id}`</code>	<code>`200 OK`</code>	ดึงข้อมูลรายละเอียด Thread, Messages ทั้งหมดตามลำดับ Ordinal, และ Current Case State
<code>`PATCH`</code>	<code>`/api/v1/chats/{thread_id}`</code>	<code>`200 OK`</code>	แก้ไขข้อมูล Thread เช่น การเปลี่ยนชื่อหัวข้อ (<code>`title`</code>)
<code>`DELETE`</code>	<code>`/api/v1/chats/{thread_id}`</code>	<code>`204 No Content`</code>	ลบ Thread ถาวร พร้อมลบ Messages, Runs, และ Case States ทั้งหมดด้วย Cascading Delete
<code>`POST`</code>	<code>`/api/v1/chats/{thread_id}/messages`</code>	<code>`202 Accepted`</code>	รับข้อความใหม่ สร้าง User Message + ChatRun (queued) และเริ่ม Background Processing
<code>`GET`</code>	<code>`/api/v1/chats/{thread_id}/runs/{run_id}`</code>	<code>`200 OK`</code>	ตรวจสอบสถานะการทำงานของ Run เฉพาะตัว (Polling Target สำหรับ Error Tracking)
<code>`POST`</code>	<code>`/api/v1/chats/{thread_id}/reports`</code>	<code>`201 Created`</code>	สร้าง Frozen Report Version จากประวัติแชทและ Case State ปัจจุบัน
<code>`GET`</code>	<code>`/api/v1/chats/{thread_id}/reports/{id}/pdf`</code>	<code>`200 OK`</code>	ดาวน์โหลดเอกสารรายงานการสืบสวนรูปแบบ PDF ที่ถูกเรนเดอร์แบบ Deterministic

8. การติดตั้งและการปรับใช้ (Deployment, Configuration & Roadmap)

ระบบรองรับการรันผ่าน Docker Compose โดยจัดการความลับ (Secrets) ผ่าน Doppler CLI:

- Docker Infrastructure: รันคอนเทนเนอร์ 4 ตัวพร้อมกัน ได้แก่ ``postgres`` (ฐานข้อมูลสัมพันธ์), ``rag-service`` (GraphRAG Microservice), ``backend`` (FastAPI Orchestrator), และ ``frontend`` (Next.js 15 Web Client)
- Secret Management: ใช้ Doppler ในการควบคุม Environment Variables ทั้งหมด (``DOPPLER_TOKEN``, ``OPENROUTER_API_KEY``, ``NEO4J_URI``, ``QDRANT_HOST``, ``DATABASE_URL``)
- Production Roadmap: การพัฒนาสู่ระดับองค์กรในอนาคตจะรวมถึง: การเพิ่ม User Authentication (JWT/OAuth2) และ Multi-tenant Row-level Security, การทำ Trash/Restore สำหรับ Chat Threads, และ Distributed Task Queue (เช่น Celery/Redis) สำหรับ Scale Worker ข้ามโฮสต์

CyberCase Intelligence Framework Technical Architecture Specification Document compiled and verified against current active repository codebase.	Status: STABLE Verified: August 2026
---	---